

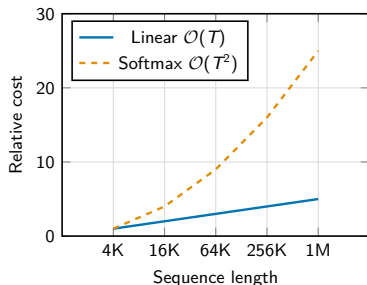
The Inference Bottleneck in Modern LLMs

Softmax attention has two scaling problems:

- Quadratic time $\mathcal{O}(T^2d)$ in sequence length T
- KV cache grows linearly with context length
- Critical for **RL test-time scaling**: long trajectories, tool-use, multi-step reasoning

Linear attention offers $\mathcal{O}(T)$ complexity but:

- Historically underperforms softmax attention
- Limited finite-state capacity for retrieval



Vanilla Linear Attention $S_t = S_{t-1} + v_t k_t^T$

Delta Rule $S_t = S_{t-1} + \beta_t \Delta v_t k_t^T$ $\Delta v_t = v_t - k_t^T S_{t-1}$

Vanilla Linear Attention $S_t = S_{t-1} + v_t k_t^T$

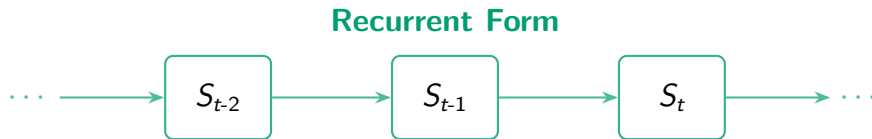
Delta Rule $S_t = S_{t-1}(I - \beta_t k_t k_t^T) + \beta_t v_t k_t^T$

GDN(Gated Delta Net) $S_t = S_{t-1}(\alpha_t(I - \beta_t k_t k_t^T)) + \beta_t v_t k_t^T$

KDA(Kimi Linear Attention) $S_t = S_{t-1}(\text{Diag}(\alpha_t)(I - \beta_t k_t k_t^T)) + \beta_t v_t k_t^T$

Challenge of Linear Attention

Hard to Parallel



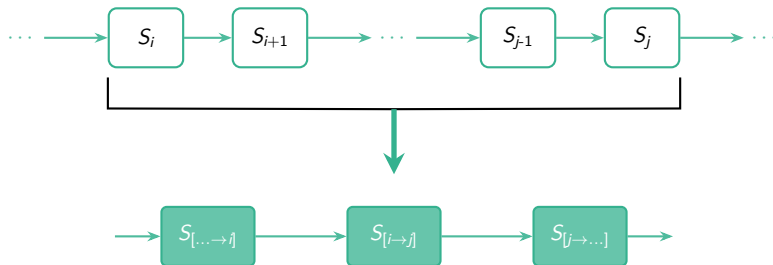
Natural in decode, but ...

Very inefficient in Prefill/Backward

Hard to apply MTP

Chunkwise Parallel Form

Chunk-wise Refactor



Inside Chunk Parallel

Outside Chunk Recurrent

Chunkwise Execution on GPU

How to run on GPU

KDA(Kimi Delta Attention)

$$\begin{aligned} S_t &= S_{t-1}(\text{Diag}(\alpha_t)(I - \beta_t k_t k_t^T)) + \beta_t v_t k_t^T \\ &= \text{Diag}(\alpha_t) S_{t-1} - \text{Diag}(\alpha_t) \beta_t k_t k_t^T S_{t-1} + \beta_t v_t k_t^T \\ &= \text{Diag}(\alpha_t) S_{t-1} + \beta_t (v_t - \text{Diag}(\alpha_t) k_t S_{t-1}) k_t^T \\ \Delta_t &= \beta_t (v_t - \text{Diag}(\alpha_t) k_t S_{t-1}) \end{aligned}$$

Chunkwise Execution on GPU

Using S_0 to represent S_t

KDA(Kimi Delta Attention)

$$S_t = \text{Diag}(\alpha_t)S_{t-1} + \Delta_t k_t^T$$

$$S_1 = \text{Diag}(\alpha_1)S_0 + \Delta_1 k_1^T$$

$$S_2 = \text{Diag}(\alpha_2 \odot \alpha_1)S_0 + \text{Diag}(\alpha_2)\Delta_1 k_1^T + \Delta_2 k_2^T$$

$$S_3 = \text{Diag}(\alpha_3 \odot \alpha_2 \odot \alpha_1)S_0 + \text{Diag}(\alpha_3 \odot \alpha_2)\Delta_1 k_1^T + \text{Diag}(\alpha_3)\Delta_2 k_2^T + \Delta_3 k_3^T$$

$$S_t = \text{Diag}(e^{G_t})S_0 + \sum_{j=1}^t \text{Diag}(e^{G_t - G_j})\Delta_j k_j^T$$

$$S_t = \text{Diag}(e^{G_t})S_0 + \sum_{j=1}^t \frac{\text{Diag}(e^{G_t})}{\text{Diag}(e^{G_j})}\Delta_j k_j^T$$

$$\Delta_t = \beta_t(v_t - \text{Diag}(\alpha_t)k_t S_{t-1})$$

$$G_i = \sum_{\ell=1}^i \log \alpha_{\ell} \in \mathbb{R}^{d_k}$$

Chunkwise Execution on GPU

Make all into Tensors

KDA(Kimi Delta Attention)

$$\Delta_t = \beta_t(v_t - \text{Diag}(\alpha_t)k_t S_{t-1})$$

$$S_t = \text{Diag}(e^{G_t})S_0 + \sum_{j=1}^t \frac{\text{Diag}(e^{G_t})}{\text{Diag}(e^{G_j})} \Delta_j k_j^T$$

$$S_t = \text{Diag}(\alpha_t)S_{t-1} + \Delta_t k_t^T$$

$$G_i = \sum_{\ell=1}^i \log \alpha_\ell$$

$$S_t = \text{Diag}(e^{G_t})S_0 + \sum_{j=1}^t \text{Diag}(e^{G_t-G_j}) \Delta_j k_j^T$$

$$A_{ij} = \beta_i k_i^T \text{Diag}(e^{G_i-G_j}) k_j, \quad j < i$$

Chunkwise Execution on GPU

Make all into Tensors

KDA(Kimi Delta Attention)

$$\Delta_t = \beta_t (v_t - \text{Diag}(\alpha_t) k_t [\text{Diag}(e^{G_{t-1}}) S_0 + \sum_{j=1}^{t-1} \text{Diag}(e^{G_{t-1}-G_j}) \Delta_j k_j^T])$$

$$= \beta_t v_t - \beta_t \text{Diag}(e^{G_t}) S_0 k_t - \sum_{j=1}^{t-1} \beta_t \text{Diag}(e^{G_t-G_j}) \Delta_j (k_j^T k_t)$$

$$A_{ij} = \beta_i k_i^T \text{Diag}(e^{G_i-G_j}) k_j, \quad j < i$$

$$S_t = \text{Diag}(\alpha_t) S_{t-1} + \Delta_t k_t^T$$
$$\Delta_t = \beta_t (v_t - \text{Diag}(\alpha_t) k_t S_{t-1})$$

$$G_i = \sum_{\ell=1}^i \log \alpha_\ell$$

$$S_t = \text{Diag}(e^{G_t}) S_0 + \sum_{j=1}^t \text{Diag}(e^{G_t-G_j}) \Delta_j k_j^T$$

$$A_{ij} = \beta_i k_i^T \text{Diag}(e^{G_i-G_j}) k_j$$

Chunkwise Execution on GPU

Make all into Tensors

$$(I + A)\Delta = BV - B \text{Diag}(e^G) K S_0^T$$

$$\Delta = (I + A)^{-1} BV - (I + A)^{-1} B \text{Diag}(e^G) K S_0^T$$

$$U = (I + A)^{-1} BV$$

$$W = (I + A)^{-1} B \text{Diag}(e^G) K$$

$$\Delta = U - W S_0^T$$

$$S_t = \text{Diag}(\alpha_t) S_{t-1} + \Delta_t k_t^T$$
$$\Delta_t = \beta_t (v_t - \text{Diag}(\alpha_t) k_t S_{t-1})$$

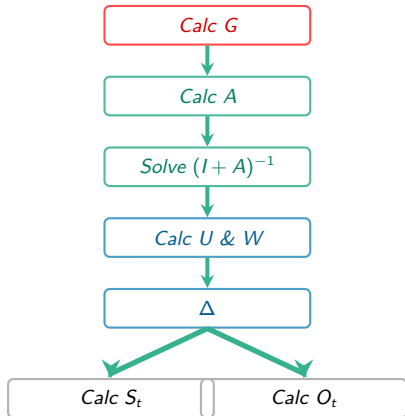
$$G_i = \sum_{\ell=1}^i \log \alpha_\ell$$

$$S_t = \text{Diag}(e^{G_t}) S_0 + \sum_{j=1}^t \text{Diag}(e^{G_t - G_j}) \Delta_j k_j^T$$

$$A_{ij} = \beta_i k_i^T \text{Diag}(e^{G_i - G_j}) k_j$$

KDA Chunkwise Forward Procedure

How to run on GPU



$$G_i = \sum_{\ell=1}^i \log_2(\alpha_\ell) \in \mathbb{R}^{d_k}$$

$$A_{ij} = \beta_i k_i^\top \text{Diag}(\exp_2(G_i - G_j)) k_j, \quad j < i$$

$$U = (I + A)^{-1} \text{Diag}(\beta) V$$

$$W = (I + A)^{-1} \text{Diag}(\beta) \text{Diag}(\exp_2(G)) K$$

$$\Delta = U - WS_0^\top$$

$$S_t = \text{Diag}(\exp_2(G_t)) S_0 + \Delta K^\top$$

$$O_t = Q_t S_t$$

KDA Kernel Implementation View

Input & Output

- Input: Q, K, V, G, β , State
- Output: S, O

vs. GDN

- Gate: scalar $\Gamma \rightarrow$ channel e^G
- Stability: division $\rightarrow$ log-domain
- Kernels: 1 $\rightarrow$ 3 (pipelined)

Kernel 1: Intra

$$\mathbf{A}_{qk} = (\mathbf{Q} \odot e^G)(\mathbf{K} \odot e^G)^\top \odot \mathbf{M}$$

$$\mathbf{A}_{kk} = \text{sLower}(\text{diag}(\beta)(\mathbf{K} \odot e^G)(\mathbf{K} \odot e^G)^\top)$$

$$\mathbf{T} = (\mathbf{I} + \mathbf{A}_{kk})^{-1}$$

} 7 MMA
+ Inverse

Kernel 2: Recomp W/U

$$\mathbf{W} = \mathbf{T}(\mathbf{K} \cdot \beta \cdot e^G)$$

$$\mathbf{U} = \mathbf{T}(\mathbf{V} \cdot \beta)$$

$$\mathbf{KG} = \mathbf{K} \cdot e^{g_{\text{last}} - G}$$

} 2 MMA
+ Elem

Kernel 3: State & Output

$$\text{NewV} = \tilde{\mathbf{U}} - \tilde{\mathbf{W}}\mathbf{S}^\top$$

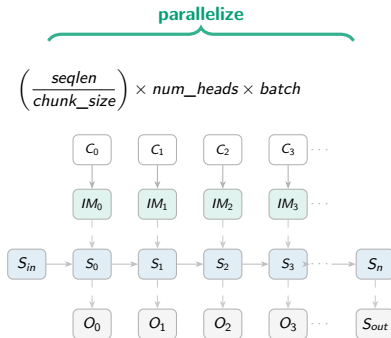
$$\mathbf{O} = \tilde{\mathbf{Q}}\mathbf{S}^\top + (\mathbf{A}_{qk} \odot \mathbf{M})\text{NewV}$$

$$\mathbf{S}' = \text{diag}(e^{g_{\text{last}}})\mathbf{S} + \text{NewV}^\top \mathbf{KG}$$

} State
recurrence

KDA (Kimi Delta Attention)

Parallelism



cuLA:

- `kda_gate_chunk_cumsum`
- `chunk_kda_fwd_intra_cuda`
- `recompute_w_u_cuda`
- `chunk_gated_delta_rule_fwd_h`
- `chunk_gla_fwd_o`

Warp-Specialized Kernel for KDA chunk_intra

Aqk + inv(Akk) with TMEM-backed UMMA pipeline

Target computation

$$A_{qk}[i, j] = s \cdot (q_i \cdot 2^{g_i})(k_j \cdot 2^{-g_j})^T, j \leq i$$

$$A_{kk}[i, j] = \beta_i(k_i \cdot 2^{g_i})(k_j \cdot 2^{-g_j})^T, j \leq i$$

$$A_{kk} \leftarrow (I - \text{tril}(A_{kk}))^{-1}$$

Input: Q, K, G, β Output: A_{qk} , A_{kk}

Warp Group Assignment

WG0 + WG1 (CudaCore): gated Q/K gen,
A→TMEM, B→SMEM, QK/KK epilogues

WG2: CollectiveInverse, A_{kk} cast & store

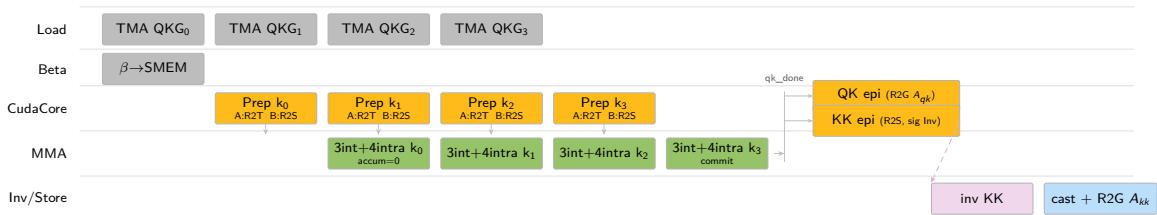
MMA warp: UMMA

3 inter + 4 intra per K iter

accumulate over 4 K iterations

Load warp: TMA Q/K/G→SMEM

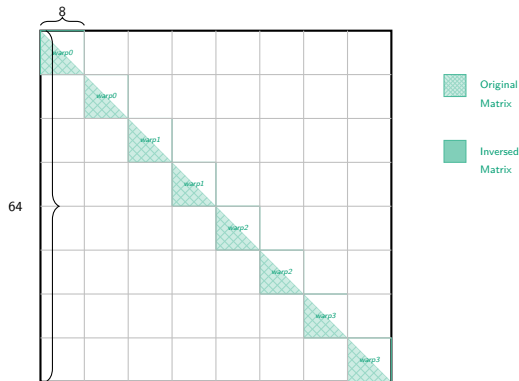
LoadBeta: β →SMEM



Matrix Inversion

Matrix Inversion for Lower Triangular Matrices — Step 1

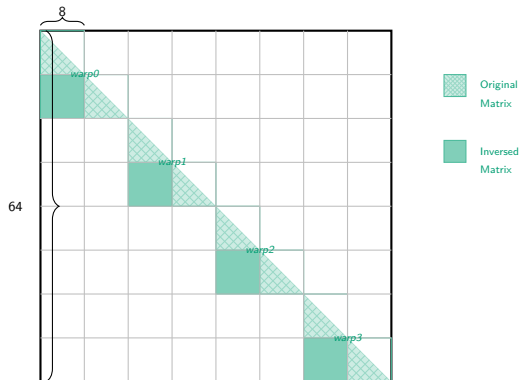
- Direct forward substitutions are easy.
 - But very expensive!
 - $O(n^2)$ instructions
 - Bad memory access pattern
 - Do it with small blocks on diagonal.
- Block-wise inverse to rescue.
 - $$\begin{bmatrix} A & 0 \\ C & D \end{bmatrix}^{-1} = \begin{bmatrix} A^{-1} & 0 \\ -(D^{-1}C)A^{-1} & D^{-1} \end{bmatrix}$$
 - Memory efficient.
 - LDSM or STSM
 - Compute efficient.
 - HMMA



Matrix Inversion

Matrix Inversion for Lower Triangular Matrices — Step 2

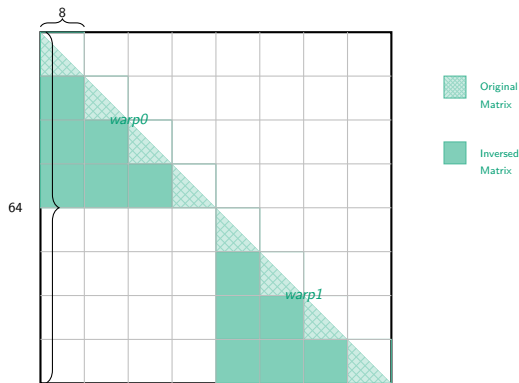
- Direct forward substitutions are easy.
 - But very expensive!
 - $O(n^2)$ instructions
 - Bad memory access pattern
 - Do it with small blocks on diagonal.
- Block-wise inverse to rescue.
 - $$\begin{bmatrix} A & 0 \\ C & D \end{bmatrix}^{-1} = \begin{bmatrix} A^{-1} & 0 \\ -(D^{-1}C)A^{-1} & D^{-1} \end{bmatrix}$$
 - Memory efficient.
 - LDSM or STSM
 - Compute efficient.
 - HMMA



Matrix Inversion

Matrix Inversion for Lower Triangular Matrices — Step 3

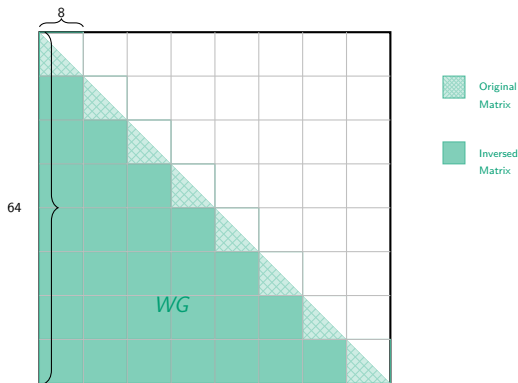
- Direct forward substitutions are easy.
 - But very expensive!
 - $O(n^2)$ instructions
 - Bad memory access pattern
 - Do it with small blocks on diagonal.
- Block-wise inverse to rescue.
 - $$\begin{bmatrix} A & 0 \\ C & D \end{bmatrix}^{-1} = \begin{bmatrix} A^{-1} & 0 \\ -(D^{-1}C)A^{-1} & D^{-1} \end{bmatrix}$$
 - Repeat formula to compute the full inverse
 - Memory efficient.
 - LDSM or STSM
 - Compute efficient.
 - HMMA



Matrix Inversion

Matrix Inversion for Lower Triangular Matrices — Final

- Direct forward substitutions are easy.
 - But very expensive!
 - $O(n^2)$ instructions
 - Bad memory access pattern
 - Do it with small blocks on diagonal.
- Block-wise inverse to rescue.
 - $$\begin{bmatrix} A & 0 \\ C & D \end{bmatrix}^{-1} = \begin{bmatrix} A^{-1} & 0 \\ -(D^{-1}C)A^{-1} & D^{-1} \end{bmatrix}$$
 - Repeat formula to compute the full inverse
 - Memory efficient.
 - LDSM or STSM
 - Compute efficient.
 - HMMA



Chunk State as Affine Transform

Foundation for CP & Intracard Parallelism

Each chunk maps its input state $\mathbf{S}$ to output state $\mathbf{S}'$ as an **Affine transform**:

$$\mathbf{S}' = \mathbf{M} \cdot \mathbf{S} + \mathbf{B}$$

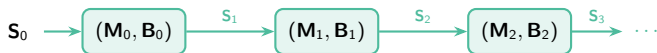
$$\mathbf{M} = \text{Diag}(\exp(\mathbf{g}_{\text{last}})) - \mathbf{K}_{\text{dec}}^{\top} \mathbf{W}$$

$$\mathbf{B} = \mathbf{K}_{\text{dec}}^{\top} \mathbf{U}$$

where $\mathbf{W}, \mathbf{U}$ from WY representation
are **independent of $\mathbf{S}$**

Composability — two chunks compose:

$$\begin{aligned}\mathbf{S}_2 &= \mathbf{M}_1 \mathbf{S}_1 + \mathbf{B}_1 \\ &= \mathbf{M}_1 (\mathbf{M}_0 \mathbf{S}_0 + \mathbf{B}_0) + \mathbf{B}_1 \\ &= \underbrace{(\mathbf{M}_1 \mathbf{M}_0)}_{\mathbf{M}_{01}} \mathbf{S}_0 + \underbrace{(\mathbf{M}_1 \mathbf{B}_0 + \mathbf{B}_1)}_{\mathbf{B}_{01}}\end{aligned}$$



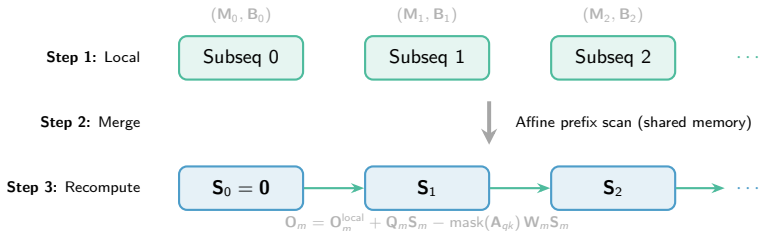
Affine composition $\Rightarrow$ **prefix scan** over chunks $\Rightarrow$ parallelism

Intracard Context Parallel

SM-level parallelism for long sequences on a single GPU

Problem: long sequence + few heads $\Rightarrow$ SM utilization too low

Solution: split into **subsequences** to saturate all SMs $\text{subseq_len} \approx \text{num_sms} \times C$



CP (Inter-Card)

Across GPUs via NCCL All-Gather

Multi-GPU training/inference

Same math: **Affine chain composition** — different granularity

Intracard CP (SM Parallel)

Within one GPU via shared memory

Single-GPU long-sequence inference

Appendix

Backup: TMEM and SMEM Placement

TMEM layout (512 cols total)

Range	Contents
[0, 64)	QK accumulator (fp32)
[64, 96)	QG_INTER (bf16)
[96, 128)	QG_INTRA (bf16)
[128, 256)	Stage 1 buffer
[256, 384)	Stage 2 buffer
[384, 512)	Padding

TMEM carries **UMMA A-operands** and accumulators (register-backed)

SMEM layout

Buffer	Contents
sQ, sK, sG	TMA-loaded input tiles
kg_all.inter	Fused K/G for inter MMAs
kg_all.intra	Fused K/G for intra MMAs
kk	Padded lower-tri for inverse
beta_smem	Beta staging buffer

SMEM carries **TMA-loaded tensors**, fused K/G B-operands, and inverse workspace

Backup: chunk_intra Optimization Highlights

1. Unified 3-stage pipeline

Q/K/G TMA load, beta load, CudaCore preparation, UMMA, and inverse are fully overlapped.

2. Gated Q/K generation on CudaCore

$q \cdot \exp_2(g - g_{\text{ref}})$ and $k \cdot \exp_2(g - g_{\text{ref}})$ computed before UMMA.

UnifiedGRef reduces redundant exp2f work.

3. Column-fused B matrix

Each K_j, G_j column read once. WG0 handles col 0+3, WG1 handles col 1+2.

4. Inter + intra UMMA schedule

3 inter MMAs + 4 intra MMAs per K iteration.

Four K iterations produce the final QK/KK accumulator.

5. Split epilogue and inverse

QK epilogue writes A_{qk} . KK epilogue prepares lower-triangular A_{kk} .

WG2 performs CollectiveInverse and stores A_{kk} .